

Decision Tree

A **decision tree** is a supervised machine learning algorithm used for both **classification** and **regression** tasks. It predicts an output by learning simple decision rules inferred from the features of the data. It works by splitting the dataset into subsets based on the feature values, forming a tree-like structure.

Structure of a Decision Tree

1. Root Node:

- Represents the entire dataset.
- Splits into branches based on a decision rule on a feature.

2. Internal Nodes:

- Represent decision points where further splits occur.
- Based on feature values, the dataset is divided into subsets.

3. Branches:

- Edges connecting nodes, representing the outcomes of the decisions.

4. Leaf Nodes:

- Terminal nodes where no further splits occur.
- Contain the final output (class or value).

How a Decision Tree Works

1. Splitting:

- The dataset is split into subsets based on decision criteria (e.g., thresholds for numerical data or categories for categorical data).

2. Selection of Splitting Attribute:

- A mathematical measure such as **Gini Index**, **Entropy** determines the best feature for splitting.

3. Stopping Conditions:

- The tree stops growing when:
 - A maximum depth is reached.

- A subset is "pure" (contains only one class or similar values).
- There are too few samples in a subset.

4. **Prediction:**

- For **classification**, the majority class in a leaf node is the predicted class.
- For **regression**, the average of target values in a leaf node is the predicted value.

Mathematical Measures for Splitting

Entropy:

- Measures the impurity or randomness in a dataset.
- Formula:

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

- P_i : Proportion of data points in class i

Information Gain (IG):

- Measures the reduction in entropy after a split.
- Formula:

$$IG(S, A) = Entropy(S) - \sum_{i=1}^k \frac{n_i}{N} \cdot Entropy(S_i)$$

- S: Original dataset.
- A: Attribute being evaluated.
- Si: Subsets after splitting by A
- ni: Number of samples in subset Si
- N: Total number of samples.

Gini Index:

- Measures impurity in a dataset (used in CART - Classification and Regression Trees).
- Formula:

$$Gini(S) = 1 - \sum_{i=1}^n p_i^2$$

Variance Reduction:

- Used in regression trees to measure the reduction in variance after a split.
- Formula:

$$VR(S, A) = \text{Variance}(S) - \sum_{i=1}^k \frac{n_i}{N} \cdot \text{Variance}(S_i)$$

Algorithm Steps

1. Initialization

- Start with the entire dataset as the root.

2. Select Splitting Attribute

- For each attribute, calculate the splitting metric (e.g., Information Gain or Gini Index).
- Choose the attribute with the best metric (e.g., highest Information Gain).

3. Split the Data

- Divide the dataset into subsets based on the chosen attribute.

4. Repeat

- Recursively apply the process to each subset.

5. Stop

- Stop splitting if:
 - All samples in a subset belong to the same class.
 - Maximum tree depth is reached.
 - Minimum subset size is reached.

6. Prediction

- Traverse the tree from the root to a leaf based on the feature values of a new instance.
- Output the class or value stored in the leaf node.

Types of Decision Trees

1. Classification Tree:

- Used for predicting categorical target variables.
- Example: Predicting if an email is spam or not.

2. Regression Tree:

- Used for predicting continuous target variables.
- Example: Predicting house prices.

Techniques to Improve Decision Trees

1. Pruning:

- Remove unnecessary branches that do not contribute significantly to accuracy.
- Types:
 - Pre-pruning: Stop splitting before the tree becomes too complex.
 - Post-pruning: Build a full tree and then remove insignificant branches.

2. Ensemble Methods:

- Combine multiple trees to improve performance:
 - **Random Forests:** Build multiple trees on random subsets of data and average their predictions.
 - **Gradient Boosting:** Build trees sequentially, where each tree corrects the errors of the previous one.

3. Hyper parameter Tuning:

- Limit tree depth, set a minimum number of samples per leaf, or specify minimum impurity reduction.

Example: Deciding Whether to Buy a House

You are deciding whether to buy a house based on the following features:

1. **Price:** Affordable, Expensive
2. **Location:** Good, Average, Bad
3. **Size:** Small, Medium, Large

The target variable is **Buy House (Yes/No)**.

Price	Location	Size	Buy House?
Affordable	Good	Large	Yes
Affordable	Average	Medium	Yes
Expensive	Good	Small	No
Expensive	Average	Medium	No
Affordable	Bad	Small	No
Expensive	Good	Large	Yes
Affordable	Good	Medium	Yes
Expensive	Bad	Large	No

Step 1: Calculate Total Entropy

The dataset contains:

- Yes = 4
- No = 4

$$Entropy(S) = -p_{Yes} \cdot \log_2(p_{Yes}) - p_{No} \cdot \log_2(p_{No})$$

$$p_{Yes} = \frac{4}{8}, \quad p_{No} = \frac{4}{8}$$

$$Entropy(S) = -\left(\frac{4}{8} \cdot \log_2 \frac{4}{8} + \frac{4}{8} \cdot \log_2 \frac{4}{8}\right)$$

$$\log_2 \frac{4}{8} = -1$$

$$Entropy(S) = -(0.5 \cdot -1 + 0.5 \cdot -1) = 1$$

Step 2: Split by Price

Price Categories:

- Affordable: 4 samples (Yes = 3, No = 1)
- Expensive: 4 samples (Yes = 1, No = 3)

Entropy of Each Subset:

1. Affordable:

$$p_{Yes} = \frac{3}{4}, \quad p_{No} = \frac{1}{4}$$

$$Entropy(Affordable) = -\left(\frac{3}{4} \cdot \log_2 \frac{3}{4} + \frac{1}{4} \cdot \log_2 \frac{1}{4}\right)$$

$$\log_2 \frac{3}{4} \approx -0.415, \quad \log_2 \frac{1}{4} \approx -2$$

$$Entropy(Affordable) \approx -(0.75 \cdot -0.415 + 0.25 \cdot -2) = 0.811$$

2. Expensive:

$$p_{Yes} = \frac{1}{4}, \quad p_{No} = \frac{3}{4}$$

$$Entropy(Expensive) = 0.811 \quad (\text{Symmetrical to Affordable})$$

Weighted Entropy for Price:

$$WeightedEntropy(Price) = \frac{4}{8} \cdot Entropy(Affordable) + \frac{4}{8} \cdot Entropy(Expensive)$$

$$WeightedEntropy(Price) = \frac{4}{8} \cdot 0.811 + \frac{4}{8} \cdot 0.811 = 0.811$$

Information Gain for Price:

$$IG(Price) = Entropy(S) - WeightedEntropy(Price)$$

$$IG(Price) = 1 - 0.811 = 0.189$$

Step 3: Split by Location

Location Categories:

- Good: 3 samples (Yes = 2, No = 1)
- Average: 2 samples (Yes = 1, No = 1)
- Bad: 3 samples (Yes = 1, No = 2)

Entropy of Each Subset:

1. Good:

$$p_{Yes} = \frac{2}{3}, \quad p_{No} = \frac{1}{3}$$

$$Entropy(Good) = -\left(\frac{2}{3} \cdot \log_2 \frac{2}{3} + \frac{1}{3} \cdot \log_2 \frac{1}{3}\right)$$

$$Entropy(Good) \approx 0.918$$

2. Average:

$$p_{Yes} = \frac{1}{2}, \quad p_{No} = \frac{1}{2}$$

$$Entropy(Average) = -\left(\frac{1}{2} \cdot \log_2 \frac{1}{2} + \frac{1}{2} \cdot \log_2 \frac{1}{2}\right)$$

$$Entropy(Average) = 1$$

3. Bad:

$$p_{Yes} = \frac{1}{3}, \quad p_{No} = \frac{2}{3}$$

$$Entropy(Bad) = 0.918 \quad (\text{Symmetrical to Good})$$

Weighted Entropy for Location:

$$WeightedEntropy(Location) = \frac{3}{8} \cdot Entropy(Good) + \frac{2}{8} \cdot Entropy(Average) + \frac{3}{8} \cdot Entropy(Bad)$$

$$WeightedEntropy(Location) = \frac{3}{8} \cdot 0.918 + \frac{2}{8} \cdot 1 + \frac{3}{8} \cdot 0.918$$

$$WeightedEntropy(Location) = 0.344 + 0.25 + 0.344 = 0.938$$

Information Gain for Location:

$$IG(Location) = Entropy(S) - WeightedEntropy(Location)$$

$$IG(Location) = 1 - 0.938 = 0.062$$

Step 4: Compare Attributes

- Price: $IG = 0.189$
- Location: $IG = 0.062$
- Size: (You can calculate similarly. Let's assume $IG(Size) = 0.100$).

Choose the Attribute with Maximum IG

- Split by Price first since $IG(Price) > IG(Location)$ and $IG(Size)$.

Step 5: Continue Splitting

For each branch of Price (Affordable, Expensive), repeat the process by calculating IG for other attributes like Location and Size. Continue until all leaf nodes are pure or stopping criteria (like max depth) are met.

Prediction Example

1. **Scenario:** Price = Affordable, Location = Good

Prediction: Yes.

2. **Scenario:** Price = Expensive, Size = Medium

Prediction: No.

IMPLEMENTATION WITH PYTHON

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt
```

```
data = {
    "Income": ["High", "Medium", "Low", "High", "Low", "Medium", "Medium",
"High"],
    "Credit Score": ["Good", "Good", "Good", "Bad", "Bad", "Bad", "Good",
"Good"],
    "Collateral": ["Yes", "Yes", "No", "Yes", "Yes", "No", "No", "No"],
    "Loan Approved": ["Yes", "Yes", "No", "No", "No", "No", "Yes", "Yes"]
}
```

```
df = pd.DataFrame(data)
df
```

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
for i in df.columns:  
    df[i] = le.fit_transform(df[i])
```

```
df
```

```
X = df[["Income", "Credit Score", "Collateral"]]  
y = df["Loan Approved"]
```

```
model = DecisionTreeClassifier(criterion="entropy", random_state=42)  
model.fit(X, y)
```

```
plt.figure(figsize=(10, 8))  
tree.plot_tree(model, feature_names=["Income", "Credit Score", "Collateral"],  
class_names=["No", "Yes"], filled=True)  
plt.show()
```

```
test_data = pd.DataFrame({  
    "Income": [le.fit_transform(data["Income"])[0]], # High  
    "Credit Score": [le.fit_transform(data["Credit Score"])[0]], # Good  
    "Collateral": [le.fit_transform(data["Collateral"])[0]] # Yes  
})
```

```
prediction = model.predict(test_data)  
print("Prediction (Loan Approved?):", le.inverse_transform(prediction)[0])
```